Comparative Analysis and Evaluation of Deep Learning Models for Automated Pork Packaging Quality Assessment (Severity Analysis)

Name: Sadul Wanasinghe
Student ID: 34964479

Research Question

Which fine-tuned object detection model is able to provide the most accurate and computationally efficient method for estimating the severity of defects in packaging for pork rashers, under varying image conditions?

The Engineering Problem

Food manufacturing and packaging factories have been increasingly transitioning away from manual inspections for their quality control to align with evolving AI technologies. These facilities require high-level quality control processes in order to correctly identify whether their products meet the necessary standard to be sent to distribution. The benefits of these technologies is that they are faster, more efficient and consistent when used in high-volume manufacturing control.

This project aims to investigate the use of fine-tuned object detection models as a way of automating the identification of packaging quality for pork rashers. By using image data, the objective is to compare and evaluate two existing detection models that will be fine-tuned to detect packaging defect severity. The performance of these models will be compared according to their accuracy as well as computational efficiency values under several image conditions, as a means to determine which is more suitable for being implemented in a real manufacturing environment.

This project is limited to only using the provided dataset, and isn't expected to be functional for other object types without additional training data or validation. The project is also limited to the provided data in dataset 3 (for example there is no data under the 'wrinkle' class in the test set).

Inputs and Outputs

Input	Format	Domain	Explanation
Pork rasher packaging images	RGB jpg: 720x540	Image data	The main input used to fine-tune the YOLO models.
Bounding box annotations	YOLO.txt	0-1	The ground truth location and size of the defects.
Defect class	Integer	0-4	Separate data into 5 defect classes.
Object-detection models	YOLOv8n/s	N/A	The existing object detection models that will be fine-tuned for the project.

Output	Format	Domain	Explanation
Detected defect class	Category	5 defect classes	The predicted defect from an image.
Defect location	Bounding box coordinates	Normalised	Predicted location of the defect.
Defect severity	Category	Low/Medium/High	Predicted severity of defect
Detection performance	Numerical	Precision, recall, mAP@0.5, mAP@0.5:0.95	Measurements for the performance of each model.
Severity performance	Numerical	Accuracy, detection rate	Measures difference between prediction and ground truth
Computational performance	Numerical	Runtime, ms/image, model size, parameter count, gpu information	Comparing the computational efficiency of each model.

Computational Solution

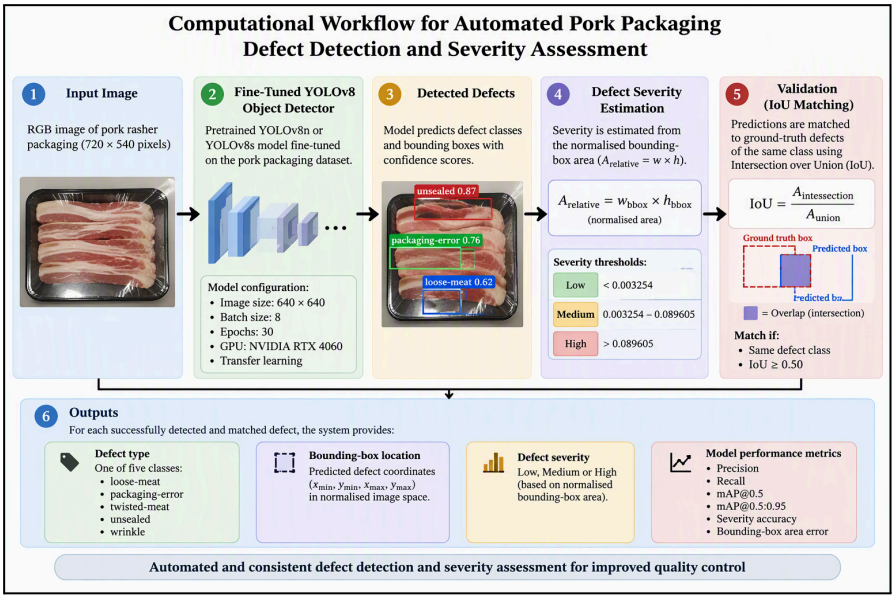
1. Dataset preparation: The pork rasher packaging dataset (dataset 3) was exported and appropriately divided into sections. The number of images was explored as well as the existing defect classes being: loose-meat, packaging-error, twisted-meat, unsealed and wrinkle. The training data was used as a way to fine-tune the existing models. The validation data was used as a way to monitor the model development. And then the test data was used to evaluate the success of the model training. From this data set preparation it was identified that there was an imbalance in defect classes, which was documented.

2. Object detection: Two models were chosen with opposite features. The YOLOv8n model is a lightweight single stage object detection model, whilst the YOLOv8s model is a larger version of the nano counterpart. This was to investigate how a larger model affects detection performance with the appropriate tradeoffs. These models weren't trained from scratch, instead were fine-tuned using the provided dataset.

3. Training configuration: Maximum epochs - 30, Image resolution - 640, Batch size - 8, Patience - 8, Random seed - 42. Both models underwent training under the same configurations, GPU and other hardware conditions.

4. Defect severity framework: By using the provided dimensions from the normalised bounding box annotations the formula A = width/height was used to estimate the defect size numerically and create a metric to label the defect severity as low, medium or high. The calculated values are as follows: low < 0.003254, 0.003254<medium<0.089605 and high >0.089605. These values were applied to the ground truth data and the predictions to allow comparison.

5. Prediction matching: Prior to comparing the predicted severity with the ground truth severity, each prediction needs to be associated with its corresponding defect. The calculation of the intersection over union (IoU) was used to do this: IoU = Aint/Auni. The prediction was only considered as being successfully matched if it was the same as the ground truth AND the IoU > 0.50. And then by using the area calculation to convert defect severity into low, medium or high, the predictions could be compared directly to the ground truth.



Alternative Solution (further explored in results)

Two object detection models from the same YOLO family were tested, including the smaller initial baseline model YOLOv8n and the larger more complicated YOLOv8s. The smaller model was chosen as the baseline due to its lightweight characteristics. The alternative was chosen due to its increased complexity whilst maintaining the same object detection architecture. Both models underwent the same parameters during fine tuning and evaluated on the same test set as well. This was to ensure that the comparison was fair, controlled and consistent.

The intention of comparing these two models was to find if there are large tradeoffs between a model's detection performance/severity evaluation and computational efficiency. During the model selection process it was inferred that larger models would be more accurate and capable for data detection, whilst being more demanding.

The goal was to compare these two models and identify which would be the ideal model type to be used in real manufacturing lines for a similar purpose.

Validation

The provided dataset contains subsets for training, validation and test data. This was found during the data exploration, completed prior to starting any model training. The training data was used in order to update the model's weights. The validation data was used only to monitor the model's performance. The test data (80 images) was used only to evaluate the model. This ensured that the model was not using images it had already seen during the training, whilst it was being evaluated. This validation strategy is using 'held-out test data'. The model's predictions were then compared against the provided ground-truths (defect class and bounding box).

The metrics used to evaluate the performance of the object detection model training was precision, recall, mAP@0.5 and mAP@0.5:0.95. The performance of the models were also tested in a per-class environment as well. This was completed due to the findings during the data exploration section of this project, revealing that there was a severe class imbalance in the provided dataset. This ensured that any skewing was identified as being caused by this imbalance.

The severity detection predictions were validated differently to the above stated. By using the IoU calculation the predictions were paired to appropriately corresponding ground truth annotations with a constraint of IoU > 0.50. If successful, these predictions were then converted into the low, medium or high severity classes by utilising the values that were calculated during the severity analysis framework section of this project. These predictions were compared to the severity from the ground truth bounding boxes. These predictions were evaluated using severity accuracy, detection rate, end-end severity and mean absolute error.

The limitations of this completed validation are that there was no wrinkle defect class data in the test set provided by the dataset. Also, there were no low-severity data, meaning the model could only be confidently evaluated for medium and high severity scenarios. The creation of the severity framework is also somewhat

flawed, due to them being approximate calculations from the bounding boxes, and isn't a pre-established known method.

Results

1. Object detection results:

Metrics	YOLOv8n	YOLOv8s
Precision	85.29%	77.11%
Recall	33.45%	43.83%
mAP@0.5	42.06%	52.09%
mAP@0.5:0.95	20.63%	26.68%

These results highlighted interesting variations in the key outputs that were being tested. YOLOv8n had around an 8% higher precision score, whilst the s model had over 10% higher recall, mAP@0.5 and mAP@0.5:0.95. This indicates that the n model was stronger at obtaining correct detections, whilst the s model was stronger at overall detection. This was an interesting result, as it was inferred that the larger model would be superior to the baseline in all aspects apart from computational speed and efficiency. Additionally both models performed the best on unsealed images, and worst on the minority classes.

2. Severity assessment results:

Metrics	YOLOv8n	YOLOv8s
Matched defects	56/79	57/79
Severity accuracy	94.64%	96.49%
Detection rate	70.89%	72.15%
End to end severity	67.09%	69.62%
Mean absolute error	0.010867	0.010055

Both models were fairly successful in identifying the severity of the defect once a prediction was correctly matched to the ground truth. However both models weren't as great at the actual detection of the defects at around 70%/ The end to end severity measurement is significantly lower

and displays a more accurate representation of the abilities of both models. This data shows that both models struggled due to detection rate, when compared to the accuracy of the detections.

3. Severity confusion matrices:

Ground Truth \ Predicted	Low	Medium	High
Low	0	0	0
Medium	0	24	2
High	0	1	29

Ground Truth \ Predicted	Low	Medium	High
Low	0	0	0
Medium	0	24	2
High	0	0	31

***

More results will be explored in the following sections.

***

Optimisation

A targeted optimization was completed on the YOLOv8s model using a lower learning rate but only for 10 epoch. This generated interesting results:

Metrics	**Baseline YOLOv8s**	**Optimised YOLOv8s**
Precision	77.11%	48.95%
Recall	43.83%	53.82%
mAP@0.5	52.09%	51.91%
mAP@0.5:0.95	26.68%	27.27%

This model was selected due to being slightly (subjectively) better at the detection and severity analysis performance. The optimisation showed improvements in the models ability to detect defects however had a significant decrease in precision. This demonstrates no strong idea through the optimisation and should be completed again, potentially at a similar 30 epoch level.

Computational performance

Numerical values found using a benchmark:

Metric	**YOLOv8n**	**YOLOv8s**
Parameters	3.14 million	11.14 million

Checkpoint size	6.19 MB	21.46 MB
Inference time	21.39 ms/image	22.45 ms/image
Duration	58min	49min

The n model was considerably smaller in size and less complex. However this didn't correlate to a large difference in inference time. Matter of fact the larger model completed the fine tuning 9 minutes faster than the smaller model. However this cannot be used as an accurate representation of computational performance as both models were not tested at the same time.

	Model	Average GPU Usage (%)	Peak GPU Usage (%)	Average VRAM (MB)	Peak VRAM (MB)	Average Temperature (C)	Peak Temperature (C)	Average Power (W)	Peak Power (W)	Benchmark Time (s)
0	YOLOv8n	31.98	76	5033.52	5035.85	40.44	41	14.61	21.92	9.57
1	YOLOv8s	37.52	68	5045.76	5047.85	46.26	49	37.30	44.74	9.21

Conclusion/Final recommendation

Overall from the results seen throughout the project the YOLOv8s model proved to be more successful in the context of being used in a real manufacturing environment. The tradeoff for a larger and more complex came at small differences in computational efficiency and generated more satisfactory results for the limited data that was tested upon. The optimisation strategy seemingly failed, representing that the originally trained YOLOv8s model is the superior option from all completed tests.

Limitations

One of the biggest limitations as discussed before was the severe imbalance in the test data provided by the dataset. Majority of the images were unsealed or packaging-error, with loose-meat and twisted-meat being minorities. Wrinkle class was completely void in the test data. This indicates that the achieved results are more successful for the abundant data, and unreliable for the sparse data.

Low severity predictions can't be successfully validated as there were 0 of these defect severity types in the matched test detections for both models. Hence the results are unreliable for this severity type.

Another large limitation with this project is that both object detection models are from the same family YOLO. Although this has its benefits, being able to compare similar model architectures, it proves a flaw in the comparison section of this project. Although the scale of the models are vastly different, the project would potentially have much clearer distinctions if another model such as R-CNN was used, or even a third YOLO model.

Finally both models were only trained on a singular data set and any overall statements on which model is superior should be considered generalisation.

Lessons

With the guidance of AI, I found that completing a thorough exploration of the dataset prior to undergoing any major development assisted in my understanding of why certain results occurred. I was able to understand why certain values seemed skewed or different than what was expected, purely because I understood the data and what it actually contained.

Accuracy cannot be used independently as a metric of performance evaluation. This is because factors such as class imbalance, test data, etc can change how a model reacts and outputs its answers. Machine learning systems are complex and shouldn't be evaluated on surface level metrics.

Machine learning projects require understanding of not only the outputs and results of the computational solution, but also the data that is being used to conduct the actual project. This allows the project to extend into further exploration, potentially using other data sets.

AI use reflection

During the completion of this project the following AI tools were used: Microsoft Copilot premium and grammarly.

Microsoft Copilot was used to understand the project brief and generate ideas for potential directions that can be completed. Copilot was also used to generate all major code that is used throughout the notebooks present in this project. Copilot was also used to keep track of progress on the project and give suggestions on what should be completed next. Copilot assisted in understanding new and difficult concepts such as the idea behind YOLO model detection models and how they are used, trained and fine-tuned. Copilot was also used to help understand and guide the creation of the GIThub repository and the contents that filled it.

The use of Copilot allowed the project to be completed with limited understanding of the YOLO model detection architecture and coding experience. Copilot also assisted in demonstrating whether the brief was being followed throughout all sections of the project to ensure all tasks were completed and within scope for this unit.

All code generated by Copilot was done so using informed, complex instructions on what I desired to advance the progress of the project. For example, Copilot suggested completing a data exploration prior to starting the project, hence I asked the AI to generate some code to identify the key features of the dataset and how it is distributed. All outputs of the generated code were personally checked to ensure it matched the desired output for the project. All generated comments and annotations were checked, and the majority was edited and changed to better match my understanding of the code's purpose.

I also ensured that I was questioning unexpected results. A big example of this was during the fine tuning of the YOLOv8n model, the provided code was only running using my laptop's CPU. Through a collaborative effort a new environment was created and used in VScode to enable GPU usage to run the model training, significantly cutting down run time for both models.

Grammarly was used to support spelling and grammar errors in the written sections of this project.

References

1. **Dataset**

@misc{ pork-rasher-error-packaging_dataset,
 title = { Pork Rasher Error(Packaging) Dataset },
 type = { Open Source Dataset },
 author = { Hello },
 howpublished = { \url{ https://universe.roboflow.com/hello-2aqe0/pork-rasher-error-packaging } },
 url = { https://universe.roboflow.com/hello-2aqe0/pork-rasher-error-packaging },
 journal = { Roboflow Universe },
 publisher = { Roboflow },
 year = { 2025 },
 month = { may },
 note = { visited on 2026-10-25 },
}

2. PyTorch (2026) *PyTorch Documentation*. Available at: PyTorch Documentation (Accessed: 22 October 2026)
3. NVIDIA (2026) *CUDA Toolkit Documentation*. NVIDIA Documentation. Available at: NVIDIA CUDA Documentation (Accessed: 22 October 2026)
4. Ultralytics (2026) *Explore Ultralytics YOLOv8*. Ultralytics Documentation. Available at: Ultralytics Docs (Accessed: 22 October 2026)